



TÓPICOS EM INFORMÁTICA

Comunicação pela rede (libgdx)



NETWORK DA LIBGDX

- A libgdx já implementa uma série de tratativas para **comunicação de rede**, sendo elas:
 - Cross-platform **HTTP requests**.
 - **Socket** (não funciona no GWT) – (é igual ao socket do Java).
 - Acesso cross-platform **ao browser**.
- Para **disparar o browser** independente da plataforma, você pode chamar o comando:
`Gdx.net.openURI(String URI);`



REQUEST HTTP

- Para enviar um HTTP request:

```
HttpRequestBuilder requestBuilder = new HttpRequestBuilder();  
HttpRequest httpRequest =  
requestBuilder.newRequest().method(HttpMethods.GET).url("http://www.google.com").build();  
Gdx.net.sendHttpRequest(httpRequest, httpResponseListener);
```

- Para fazer um HTTP com argumentos:

```
HttpRequestBuilder requestBuilder = new HttpRequestBuilder();  
HttpRequest httpRequest =  
requestBuilder.newRequest().method(HttpMethods.GET).url("http://www.google.com").content("q=  
libgdx&example=example").build();  
Gdx.net.sendHttpRequest(httpRequest, httpResponseListener);
```



OBSERVAÇÕES

- Sockets **não funciona** no GWT (versão browser da libgdx).
 - A alternativa são **websockets** mas a libgdx ainda não suporta. Dá pra fazer na mão caso necessário mas **não é tão viável** porque não é tão rápido quanto o socket padrão.
- No android **pra acessar a rede** (internet) precisa adicionar a permissão no *AndroidManifest*:
 - `<uses-permission android:name="android.permission.INTERNET" />`
- No android **não dá** pra acessar a rede na thread principal.
- Tomar cuidado com o **acesso à rede** pois **consome bateria** principalmente em dispositivos mobile.



SOCKET

- A libgdx **também fornece** funções para fazer comunicação por socket.
 - Contudo, a comunicação é **baseada no socket do próprio Java**.
 - Dessa forma, vamos utilizar **diretamente o socket do Java** ao invés do wrapper da libgdx.



OBSERVAÇÕES

- Para colocar de fato o servidor funcionando pela internet você precisa:
 1. Fazer o **encaminhamento de portas** no seu roteador. Escolha uma porta e encaminhe para o IP local do PC dentro da sua rede que está rodando o servidor do jogo.
 2. Em alguns casos, **liberar a porta escolhida no firewall** do seu sistema operacional.
- É possível pagar por um ip global estático ou ip dinâmico como
 - <https://www.noip.com/>

SERVIDOR DE AUTENTICAÇÃO

```
//Cria um novo socket de servidor na porta especificada
serverSocket = new ServerSocket(PORT);

while (true) {
    in = null;
    out = null;

    System.out.println("Waiting...");
    //Espera com que uma conexão seja feita e retorna esse socket
    socket = serverSocket.accept();

    in = new ObjectInputStream(socket.getInputStream());

    //Lê o objeto que veio de entrada
    accountIn = (Client$AccountLogin) in.readObject();

    //Busca a conta no banco de dados
    Server$Account accResponse =
        database.specific.AccountsDatabase.read(accountIn.getAccount());

    //Confere o password
    boolean accountFound = false;
    if (accResponse != null)
        accountFound = accountIn.getPassword().equals(accResponse.getPassword());

    // Initializing an AccountReply to the client
    out = new ObjectOutputStream(socket.getOutputStream());
```

```
// Verificação de conta
if (accountFound) { //Conta encontrada
    accResponse.setAccountFound(true);

    //Cria a thread de conexão (faz um novo socket dentro dessa classe)
    c = new Connection(accResponse);
    c.setName("Connection [ACCOUNT: " + accResponse.getAccount() + ", IP: " +
        socket.getInetAddress() + "]");
    ConnectionManager.getCurrentConnections().put(accResponse.getCharacterId(), c);

    //seta o IP e a PORTA
    accResponse.setServerIP(GAME_IP);
    accResponse.setServerPort(c.getPort());

    c.start();

    //flush
    out.writeObject(accResponse);
    c = null;
} else { //Conta não encontrada
    out.writeObject(new Server$Account(false, false));
}
```

CONEXÃO

- A classe de conexão (que vai manter a conexão com o servidor e cliente), inicializa um socket da seguinte forma:

```
protected void startConnection(Server$Account ar){
    try {
        // Cria um novo server socket
        serverSocket = new ServerSocket(0); //0 = "random" port

        this.unblock();

    } catch (Exception ex) {
        System.out.println(this.getName() + " - Error occurred while creating the
server socket on port "
            + serverSocket.getLocalPort() + ": " + ex.getMessage());
        return;
    }
}

public void run() {
    //Inserir a nova conexão startada na lista de conexões
    ConnectionManager.insert(this);

    While:
    while (true){
        while (blocked)
            continue While;
    }
}
```

...

```
public void send(Object o){
    if (blocked || socket == null) return;

    try {
        if (out == null) {
            out = new ObjectOutputStream(new
BufferedOutputStream(socket.getOutputStream()));
        }
        out.writeObject(o);
        out.flush();
    } catch (IOException e) {
        System.out.println(this.getName() + " - Error on thread while sending
object.");
        e.printStackTrace(System.out);
    }
}
```




CLIENTE

- O cliente se conecta da seguinte forma:

```
private static boolean connect(){
    if (connected) return false;

    try {
        socket = new Socket(LISTENER_IP,
LISTENER_PORT);
        connected = true;
        in = null; out = null;
        return true;
    } catch (Exception ex) {
        System.out.println("Error connecting to
server: " + ex.getMessage());
        return false;
    }
}
```

```
public static boolean send(Object o){
    try{
        connect();
        if (out == null) {
            out = new ObjectOutputStream(new
BufferedOutputStream(socket.getOutputStream()));
        }
        out.reset();
        out.writeObject(o);
        out.flush();
        return true;

    }catch (IOException e){
        System.out.println("Error while sending to the server: " +
e.getMessage());
        return false;
    }
}
```